SYSTEM ARCHITECTURE DOCUMENT

Real-Time Security Log Anomaly Detection and LLM-Based Incident Triage

Version 1.0
April 2026

Team:
Aboobakker Twaha (Lead) | Adwaith Shine | Dhruvnand T | Karthik Rajeevan

External Guide: Prof. Mahendra Pratap Singh (NITK Surathkal)
Internal Guide: Mrs. Anusha Anchan (AJIET)

1. System Overview

This document defines the complete technical architecture for the Real-Time Security Log Anomaly Detection and LLM-Based Incident Triage system. It is designed as a reference for all team members and covers every component, technology choice, data flow, and configuration parameter. If something is not covered here, ask before implementing.

1.1 Purpose

The system is an automated first-level threat response tool for Security Operations Center (SOC) workflows. It ingests security logs, detects anomalous patterns using deep learning (NeuralLog), and generates structured incident triage reports using a Large Language Model (LLM). It does not replace human analysts. It reduces their manual triage burden by producing actionable reports for flagged anomalies.

1.2 What the System Does End-to-End

1. Ingests security logs from file-based sources (simulating real-time streaming)
2. Preprocesses log messages: tokenizes, lowercases, removes non-character tokens
3. Constructs log sequences using a sliding window approach
4. Routes sequences through a two-lane async queue (standard path vs critical fast-path)
5. Runs NeuralLog (BERT + Transformer encoder) anomaly detection on sequences
6. Escalates anomalous sequences to an LLM-based triage agent
7. The LLM agent enriches context using four tools: CVE lookup, IP reputation, historical pattern similarity search, and MITRE ATT&CK technique mapping
8. Generates a structured JSON triage report with severity, description, evidence, remediation, and MITRE mapping
9. Stores reports in Qdrant (vector DB) for both retrieval and future similarity search
10. Displays alerts and reports on a Next.js dashboard served by a FastAPI backend

2. Technology Stack

Every technology choice below is final unless Prof. Singh requests a change.

Component	Technology	Why This Choice
Language	Python (backend/pipeline), TypeScript (frontend)	Team proficiency, ecosystem support for ML/NLP
Anomaly Detection (Primary)	NeuralLog (Le & Zhang, ASE 2021)	Parsing-free, strong chronological split performance, BERT semantic embeddings

Anomaly Detection (Baseline)	DeepLog LSTM	Standard benchmark baseline for comparison only
Deep Learning Framework	PyTorch	NeuralLog reference implementation uses PyTorch
BERT Embeddings	bert-base-uncased (via HuggingFace transformers)	Used inside NeuralLog for log message semantic representation
Report Embeddings	all-MiniLM-L6-v2 (via sentence-transformers)	384-dim vectors optimized for semantic similarity of triage reports
Streaming / Queue	Redis Streams	Lightweight, consumer groups, persistence, replaces Kafka
LLM (Remote)	Groq-hosted Llama 3 via LangChain	Free tier API, fast inference, tool use support
LLM (Local Fallback)	Ollama with Gemma 3 4B / Mistral 7B / Llama 3.2 3B	Fallback when Groq rate limits are hit
LLM Orchestration	LangChain	Agent framework with tool use, supports both Groq and Ollama backends
Vector Database	Qdrant (Docker container)	Standalone server, payload filtering, stores both vectors and full report JSON
Backend API	FastAPI	Async-native, OpenAPI docs, WebSocket support for real-time streaming
Frontend / Dashboard	Next.js	React-based, SSR support, modern component ecosystem
External APIs	NVD API (CVE lookup), AbuseIPDB (IP reputation)	Both have free tiers sufficient for demo
MITRE ATT&CK Data	Local JSON from MITRE STIX/TAXII or GitHub	Offline lookup, no API dependency

3. Datasets and Their Roles

3.1 HDFS (ML Evaluation)

Source: Loghub (Zenodo), originally collected from Hadoop cluster on Amazon EC2

Size: 11,175,629 log messages

Anomaly Rate: 2.93%

Anomaly Type: Infrastructure failures (block replication issues)

Sequence Grouping: By block ID (each HDFS block has a sequence of related log entries)

Role: Primary NeuralLog training and evaluation dataset. Standard benchmark with published results for direct comparison.

3.2 BGL (ML Evaluation)

Source: Loghub (Zenodo), collected from BlueGene/L supercomputer

Size: 4,747,963 log messages

Anomaly Rate: 7.34%

Anomaly Type: Hardware/system failures

Sequence Grouping: Chronological sliding window (size 20, step 1)

Role: Secondary benchmark. Harder dataset. Used to validate chronological split performance (NeuralLog F1 0.98 vs DeepLog F1 0.43).

3.3 AIT-LDS v2.0 (SOC Demonstration + LLM Triage)

Source: Austrian Institute of Technology, Zenodo (DOI: 10.5281/zenodo.5789064)

License: CC BY-NC-SA 4.0 (free for academic use)

Structure: Eight testbed scenarios, each simulating a small enterprise network (mail server, file share, WordPress, VPN, firewall)

Duration: 4-6 days per scenario with simulated normal user behavior + multi-step attack

Log Types: auth.log, audit.log, syslog, DNS logs, VPN logs, Apache logs, Suricata logs

Attack Types: Network scans (Nmap), directory scans (Dirb), WPScan, webshell upload, password cracking, reverse shell, privilege escalation, DNS data exfiltration

Labels: Fine-grained, per-line labels mapping each log event to specific attack steps

Role: Feeds labeled attack events to the LangChain triage agent. Produces security-relevant triage reports (not Hadoop failures). Also used for the transfer learning stretch goal.

3.4 Dropped Datasets

UNSW-NB15: Pre-extracted network flow features in tabular CSV. Not sequential log messages. Incompatible with NeuralLog.

ADFA-LD: System call traces as integer sequences. Not text log messages. Same incompatibility.

4. Pipeline Architecture: Five Stages

The system is a five-stage streaming pipeline. Each stage is an independent Python process communicating through Redis Streams. All stages run concurrently.

4.1 Stage 1: Log Ingestion

Component: ingestion/producer.py + ingestion/prefilter.py

Input: Raw log files from datasets (HDFS, BGL, AIT-LDS v2.0)

Output: Redis Stream stream:raw_logs

The producer reads log files line by line with configurable throughput (to simulate real-time ingestion). Each log entry is assigned a timestamp and published to the Redis Stream stream:raw_logs.

Before publishing, a lightweight pre-filter runs pattern matching on each raw log line. This is not ML-based; it is simple keyword and regex matching. The pre-filter checks for:

- Log severity fields containing CRITICAL, EMERGENCY, or FATAL
- Repeated authentication failures from a single source IP within a configurable time window
- Specific high-priority event IDs: account lockout events, privilege escalation indicators (su, sudo failures), SSH brute force patterns

Logs matching any pre-filter rule get a metadata field priority: critical attached. All logs (with or without the critical tag) go to the same stream:raw_logs stream. The priority tag is used downstream in Stage 2 for routing.

Why pre-filter at ingestion? Some events are so obviously critical (e.g., a CRITICAL severity syslog entry about a kernel panic) that waiting for NeuralLog to score them would add unnecessary latency. The pre-filter ensures these reach the LLM triage agent immediately. Everything else still goes through NeuralLog.

4.2 Stage 2: Preprocessing and Sequence Construction

Component: preprocessing/sequence_builder.py + dataset_loaders/

Input: Redis Stream stream:raw_logs

Output: Redis Streams stream:queue_a or stream:queue_b

A consumer reads from stream:raw_logs. For NeuralLog, preprocessing is minimal because it is parsing-free:

1. Tokenize each log message into word tokens using standard delimiters (whitespace, colon, comma)
2. Convert all characters to lowercase
3. Remove all non-character tokens (numbers, special characters, operators, punctuation)

Sequences are then constructed using a sliding window:

- Window size: 20 log messages (NeuralLog paper default)
- Step size: 1

- HDFS: grouped by block ID
- BGL: grouped chronologically
- AIT-LDS v2.0: grouped by host and log file, chronologically

Routing decision: if any log message within the constructed sequence has the priority: critical tag from Stage 1, the entire sequence is published to stream:queue_b. Otherwise, it goes to stream:queue_a.

4.3 Stage 3: Anomaly Detection (Two-Lane Queue)

This is the core of the pipeline. Two independent workers consume from two separate Redis Streams, running fully in parallel with zero dependency on each other.

4.3.1 Queue A Worker (Standard Path)

Component: queue/queue_a_worker.py

Reads from: stream:queue_a

Model: NeuralLog (loaded once at startup, kept in memory)

The worker reads sequences from stream:queue_a and runs NeuralLog inference. NeuralLog produces a binary classification (normal/anomalous) with a confidence score. The model is served as a persistent Python process with the trained PyTorch model loaded in GPU/CPU memory. No separate model serving infrastructure (like TorchServe) is needed at demo scale.

If the sequence is classified as anomalous AND the confidence score exceeds a configurable threshold (default: 0.5, tuned during evaluation):

- The detection result (original log sequence + anomaly score + classification) is published to stream:queue_b_escalated

If the sequence is classified as normal:

- The result is published to stream:results_normal for dashboard logging (low priority, used for statistics only)

4.3.2 Queue B Worker (Critical Fast-Path)

Component: queue/queue_b_worker.py

Reads from: stream:queue_b (pre-filter critical events) AND stream:queue_b_escalated (NeuralLog escalations)

Queue B receives items from two sources:

1. Pre-filter critical events (from Stage 1/2): These bypassed the Queue A scoring path. The Queue B worker still runs NeuralLog inference on these to get an anomaly score (for inclusion in the triage report), but proceeds to LLM triage regardless of the score.
2. NeuralLog escalations (from Queue A): These already have an anomaly score attached. No need to re-run NeuralLog.

All items reaching Queue B proceed to Stage 4 (LLM Triage). This is the only path to the LLM.

Critical design principle: Queue A and Queue B are fully independent. If Queue A crashes, Queue B continues to process pre-filter critical events. If Queue B is slow (due to LLM latency), Queue A continues to score sequences. There is no timeout, no shared state, and no dependency between them.

4.4 Stage 4: Context Enrichment + LLM Triage Report Generation

Component: triage/agent.py + triage/tools/ + triage/prompts/

The LangChain agent receives the anomalous log sequence, anomaly score, and metadata (source host, log type, timestamp range) from the Queue B worker.

4.4.1 LLM Backend Configuration

The LLM backend is configurable in settings.yaml:

Mode	Provider	Model	Tool Use Strategy
Remote (default)	Groq API	Llama 3 8B/70B	ReAct agent: LLM decides which tools to call
Local (fallback)	Ollama	Gemma 3 4B / Mistral 7B / Llama 3.2 3B	Deterministic: all tools called before LLM invocation, results injected into prompt

Why the different strategies? Small local models (4B-7B parameters) are unreliable at function calling / tool use. They may fail to produce valid tool call JSON or may call the wrong tool. To avoid this, when using a local model, all four tools are called deterministically before the LLM is invoked, and the results are injected directly into the prompt. The LLM then only needs to write the triage report using the pre-fetched context. When using Groq (which hosts larger models with better tool use capabilities), the LLM decides which tools to call using a ReAct loop.

4.4.2 Agent Tools

Tool 1: CVE Lookup (NVD API)

- Input: software name/version or vulnerability keyword extracted from log context
- Output: matching CVE entries with severity scores (CVSS), descriptions, and affected products
- API: National Vulnerability Database (NVD) REST API. Free tier, rate limited to 5 requests per 30 seconds without API key, 50/30s with API key.
- When used: when log messages reference specific software names, service versions, or error patterns suggesting a known vulnerability

Tool 2: IP Reputation (AbuseIPDB API)

- Input: IP address extracted from the log
- Output: abuse confidence score (0-100), country, ISP, number of recent reports, categories of abuse
- API: AbuseIPDB v2 REST API. Free tier: 1000 checks/day.
- When used: when log messages contain external/source IP addresses (SSH brute force sources, scanner IPs, suspicious connection origins)

Tool 3: Historical Pattern Lookup (Qdrant Vector Search)

- Input: text summary of the current anomalous log sequence
- Processing: embed the input using all-MiniLM-L6-v2, search Qdrant for top-3 most similar past triage reports by cosine similarity
- Output: similar past incident IDs, their severity levels, descriptions, and remediation steps
- When used: always called to check if this type of anomaly has been seen before

Tool 4: MITRE ATT&CK Technique Mapper

- Input: keywords/patterns from the anomalous log (e.g., su, PAM:authentication, nmap, failed password)
- Processing: matches against a local JSON file containing 30-50 MITRE ATT&CK technique entries relevant to the project scope
- Output: matching ATT&CK technique ID, technique name, tactic name, and description
- Data source: downloaded from MITRE ATT&CK STIX/TAXII feed or their GitHub repository. Stored locally, no API calls needed.
- When used: always called to map the anomaly to a known adversary technique

4.4.3 Triage Report Schema

The LLM generates a structured JSON report with the following fields:

Field	Type	Description

incident_id	UUID (auto-generated)	Unique identifier for this incident
timestamp	ISO 8601 string	When the anomaly was detected
severity	Enum	Critical | High | Medium | Low | Informational
title	String	One-line incident summary
affected_host	String	Hostname or IP of the affected system
log_source	String	Which log file the anomaly originated from (auth.log, syslog, audit.log, etc.)
anomaly_score	Float (0.0-1.0)	NeuralLog confidence score
description	String	Multi-sentence contextual explanation of the anomaly
evidence	Array of strings	Relevant raw log lines that triggered the detection
cve_references	Array of strings	CVE IDs if applicable (from Tool 1)
ip_reputation	Object	IP, abuse score, country (from Tool 2)
mitre_attack	Object	Technique ID, technique name, tactic (from Tool 4)
remediation_steps	Array of strings	Ordered list of recommended actions
similar_past_incidents	Array of strings	Incident IDs of similar past events (from Tool 3)

Reports are stored in Qdrant with the description + title + remediation concatenated and embedded as the vector, and the full JSON report stored as the payload. This means Qdrant serves as both the vector store for Tool 3 and the primary report database for the dashboard.

4.5 Stage 5: Alert Dashboard

Frontend: Next.js

Backend API: FastAPI (Python)

Data Source: Qdrant (for report retrieval) + Redis Streams (for real-time feed via WebSocket)

The dashboard has four views:

1. Real-Time Alert Feed: A live-updating list of triage reports sorted by severity (Critical first). Uses WebSocket connection from FastAPI to push new reports as they are generated. Each entry shows severity badge, title, affected host, timestamp, and anomaly score.

2. Report Detail View: Clicking on any alert opens the full triage report. Shows all JSON schema fields in a readable layout: severity classification, description, raw evidence log lines, CVE references (if any), IP reputation results, MITRE ATT&CK mapping, remediation steps, and links to similar past incidents.

3. Timeline View: A chronological view showing anomaly detections over time. X-axis is time, Y-axis is severity. Helps visualize attack patterns and burst activity.

4. Summary Statistics: Total alerts, severity distribution (pie/bar chart), top affected hosts, most common MITRE ATT&CK techniques detected, detection rate over time.

5. Redis Streams Configuration

All inter-process communication uses Redis Streams with consumer groups. This gives us message persistence, acknowledgment (failed processing retries), and the ability to run multiple consumers.

Stream Name	Producer	Consumer	Consumer G
stream:raw_logs	ingestion/producer.py	preprocessing/sequence_builder.py	group:preproc
stream:queue_a	preprocessing/sequence_builder.py	queue/queue_a_worker.py	group:queue_
stream:queue_b	preprocessing/sequence_builder.py	queue/queue_b_worker.py	group:queue_
stream:queue_b_escalated	queue/queue_a_worker.py	queue/queue_b_worker.py	group:queue_
stream:results_normal	queue/queue_a_worker.py	dashboard (FastAPI)	group:dashbo
stream:triage_reports	queue/queue_b_worker.py	dashboard (FastAPI)	group:dashbo

Redis setup: single Redis server instance (Docker or local install). No cluster mode needed. Default port 6333. Configuration in config/settings.yaml.

6. Qdrant Vector Database

Deployment: Single Docker container: docker run -p 6333:6333 qdrant/qdrant

Python Client: qdrant-client library

Embedding Model: all-MiniLM-L6-v2 (384 dimensions)

6.1 Collection Schema

One collection named triage_reports with the following configuration:

- Vector size: 384 (matching all-MiniLM-L6-v2 output)
- Distance metric: Cosine

- Payload: full triage report JSON (all fields from the schema in Section 4.4.3)

6.2 Ingestion Flow

1. LLM generates triage report JSON
2. Concatenate title + description + remediation_steps into a single text block
3. Embed the text using all-MiniLM-L6-v2
4. Upsert into Qdrant: vector = embedding, payload = full report JSON, id = incident_id UUID

6.3 Query Flow (Tool 3)

1. Receive current anomalous log sequence text
2. Embed using all-MiniLM-L6-v2
3. Search Qdrant for top-3 nearest neighbors by cosine similarity
4. Return matched reports with their severity, description, and incident_id

6.4 Dashboard Queries

FastAPI queries Qdrant directly using payload filtering for dashboard features: filter by severity level, filter by time range (using timestamp in payload), filter by affected host, retrieve specific report by incident_id. This eliminates the need for a separate relational database.

7. Project Directory Structure

All code lives in a single repository. The structure below shows every file and what it does.

```
soc-triage/
  config/
    settings.yaml              # All configurable parameters
  ingestion/
    producer.py                 # Reads log files, publishes to Redis
    prefilter.py                # Keyword/pattern rules for critical tagging
  preprocessing/
    sequence_builder.py         # Sliding window, tokenization
    dataset_loaders/
      hdfs_loader.py            # HDFS grouping by block ID
      bgl_loader.py             # BGL chronological grouping
      ait_loader.py             # AIT-LDS per-host, per-logfile
  detection/
    neurallog/
```

```
    model.py                    # NeuralLog architecture
    train.py                    # Training script
    inference.py                # Inference for pipeline
    embeddings.py               # BERT embedding extraction + caching
  deeplog/
    model.py                    # DeepLog LSTM (baseline only)
    train.py
    inference.py
triage/
  agent.py                      # LangChain ReAct agent setup
  tools/
    cve_lookup.py               # NVD API tool
    ip_reputation.py            # AbuseIPDB API tool
    historical_lookup.py        # Qdrant vector search tool
    mitre_mapper.py             # MITRE ATT&CK technique mapper
  prompts/
    triage_prompt.py            # Prompt template + output schema
  report_schema.py              # Pydantic model for report validation
queue/
  redis_manager.py              # Redis connection, consumer group setup
  queue_a_worker.py             # Standard path worker
  queue_b_worker.py             # Critical fast-path worker
  router.py                     # Routing logic
dashboard/
  backend/                      # FastAPI
    main.py                     # FastAPI app entry point
    routes/
      alerts.py                 # REST endpoints for reports
      ws.py                     # WebSocket for real-time feed
    services/
      qdrant_service.py         # Qdrant query helpers
  frontend/                     # Next.js app
    src/
      pages/                    # Next.js pages
      components/               # React components
evaluation/
  metrics.py                    # F1, precision, recall
  benchmark.py                  # NeuralLog on HDFS/BGL
  transfer_experiment.py        # AIT transfer learning (stretch)
data/
  raw/                          # Dataset files
  embeddings/                   # Cached BERT embeddings
  models/                       # Trained checkpoints
  mitre/                        # MITRE ATT&CK JSON data
scripts/
```

```
    setup_redis.sh              # Redis server setup
    setup_qdrant.sh             # Qdrant Docker setup
    download_datasets.sh        # Dataset download
    run_pipeline.sh             # Full pipeline launch
  tests/
  requirements.txt
  README.md
```

8. Key Configuration Parameters

All parameters are centralized in config/settings.yaml. These are the critical ones:

Parameter	Default Value	Notes
neurallog.window_size	20	Sliding window size, from NeuralLog paper
neurallog.step_size	1	Sliding window step
neurallog.anomaly_threshold	0.5	Confidence threshold for Queue A escalation. Tune during evaluation.
redis.host	localhost	Redis server address
redis.port	6379	Redis server port
qdrant.host	localhost	Qdrant server address
qdrant.port	6333	Qdrant REST API port
qdrant.collection	triage_reports	Collection name
llm.provider	groq	groq or ollama
llm.groq_model	llama3-70b-8192	Groq model identifier
llm.ollama_model	gemma3:4b	Ollama model name
llm.groq_api_key	(env var)	Set via GROQ_API_KEY environment variable
api.abuseipdb_key	(env var)	Set via ABUSEIPDB_API_KEY. Free tier: 1000/day.
api.nvd_rate_limit	5 req / 30 sec	Without API key. 50/30s with key.
prefilter.critical_keywords	[CRITICAL, EMERGENCY, FATAL]	Keywords triggering Queue B routing
prefilter.auth_fail_threshold	5 failures in 60 seconds	Brute force detection window
dashboard.fastapi_port	8000	FastAPI backend port
dashboard.nextjs_port	3000	Next.js frontend port

9. Transfer Learning Experiment (Stretch Goal)

Status: Do NOT start until the core pipeline (Stages 1-5) is fully working end-to-end.

The experiment tests whether a NeuralLog model trained on BGL supercomputer logs can transfer its anomaly detection capability to security-relevant logs from AIT-LDS v2.0.

Step 1 (Zero-shot): Train NeuralLog on BGL. Evaluate directly on AIT auth.log/syslog data without retraining. Expected result: poor performance (different log vocabulary/patterns).

Step 2 (Fine-tuned): Fine-tune the BGL-trained model on 50-70% of AIT data (1-2 scenarios). Evaluate on remaining scenarios. This tests whether domain-general anomaly detection knowledge transfers with minimal labeled security data.

Why this matters: Most published log anomaly detection papers only evaluate on HDFS/BGL/Thunderbird. Demonstrating transfer (or lack thereof) to security-relevant log data addresses an open gap identified in the Landauer et al. 2023 survey.

10. Key References

- NeuralLog: Le, V.-H. and Zhang, H. (2021). Log-based Anomaly Detection Without Log Parsing. ASE 2021, pp. 492-504. arXiv:2108.01955
- DeepLog: Du, M. et al. (2017). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. CCS 2017.
- LogBERT: Guo, H. et al. (2021). LogBERT: Log Anomaly Detection via BERT. IJCNN 2021.
- Landauer et al. (2023). Deep Learning for Anomaly Detection in Log Data: A Survey. DOI: 10.1016/j.mlwa.2023.100470
- LogLLM: Guan, Z. et al. (2024). arXiv:2411.08561
- LogPrompt: Liu, J. et al. (2023). arXiv:2308.07610
- Advancing Autonomous Incident Response: arXiv:2508.10677
- AIT-LDS v2.0: Landauer et al. (2023). Maintainable Log Datasets for Evaluation of Intrusion Detection Systems. IEEE TDSC, vol. 20, no. 4, pp. 3466-3482.
- Retrieval-Augmented LLMs for Security Incident Analysis: Cadet et al. (2026). arXiv:2603.18196